

System Architecture & Process Flow

Library Management System with Book Recommendation & Late-Return Risk Prediction

Key Terms

- **Architecture** — how the system's pieces (React, Express, MongoDB, Flask) are connected and communicate with each other
- **Schema** — the structure of the database: collections, fields, and relationships
- **Process flow** — the sequence of actions and decisions that happen when a user performs an action in the system

1. System Architecture

The system uses a layered architecture with the machine learning component isolated as its own service. The React frontend never talks to the ML service directly — every request passes through Express, which reads/writes MongoDB and calls the Flask service only when a prediction is needed. This keeps the MERN application functional even if the ML service is temporarily unavailable.

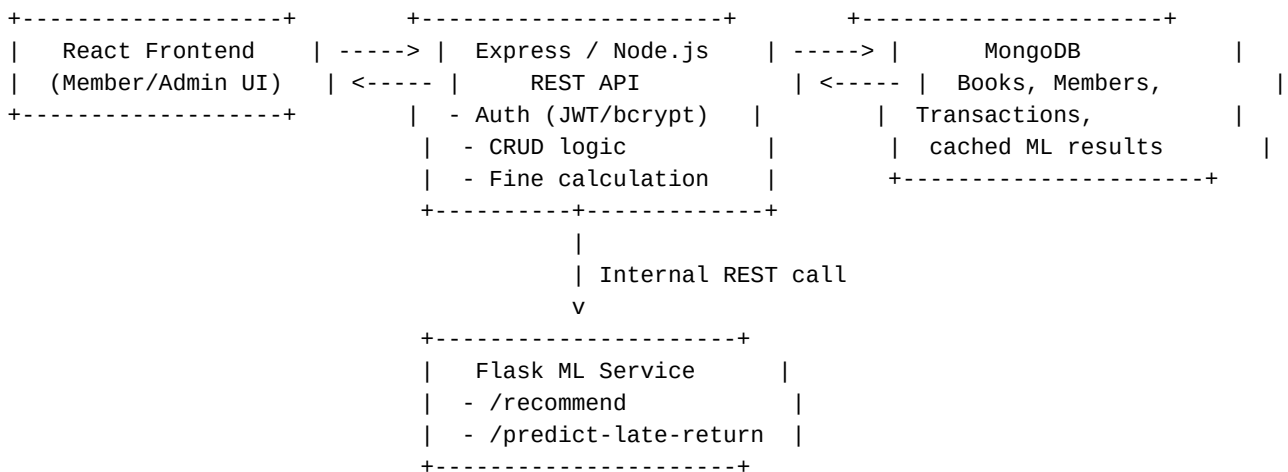


Figure 1. High-level system architecture

2. Process Flow: Authentication

- 1 User submits email and password via the React login form
- 2 Express receives the request and looks up the Member record in MongoDB by email
- 3 The submitted password is compared against the stored bcrypt hash
- 4 If valid, Express issues a JWT token containing the user's role (member / librarian / admin)
- 5 React stores the token and uses it to access role-appropriate pages and API routes

3. Process Flow: Borrowing a Book

- 1 Member selects a book and requests to borrow it
- 2 Express checks the Book collection to confirm availableCopies > 0
- 3 If available, Express creates a new Transaction record (borrowDate, dueDate, status = borrowed)
- 4 Express calls the Flask ML service at /predict-late-return, sending the member's and book's relevant features
- 5 The model returns a risk score based on the member's return history, book category, and loan duration
- 6 Express saves the risk score onto the Transaction record
- 7 If the risk is high, the transaction appears flagged on the librarian's dashboard for follow-up
- 8 Express decrements availableCopies on the Book record

4. Process Flow: Returning a Book

- 1 Librarian marks a Transaction as returned via the UI, or the member returns it in person and staff logs it
- 2 Express sets returnDate and updates the Transaction status to returned
- 3 Express compares returnDate against dueDate; if late, a Fine record is created and linked to the member
- 4 The member's lateReturnHistory count is incremented if the return was late — this feeds future risk predictions
- 5 Express increments availableCopies on the Book record
- 6 If the book has pending Reservations, the next member in the queue is notified

5. Process Flow: Book Recommendations

- 1 A member borrows or views a book, which Express logs as an interaction event
- 2 This event triggers a recommendation refresh job for that member
- 3 The Flask ML service computes similarity scores using TF-IDF over each book's genre, author, and description
- 4 The top-ranked matches are written to the Recommendation collection in MongoDB, cached per member
- 5 When the member visits their dashboard, React fetches the cached list from Express (not a live ML call) and displays it as "Recommended for you"

6. Process Flow: Reserving an Unavailable Book

- 1 Member attempts to borrow a book with availableCopies = 0
- 2 React offers the option to reserve instead of borrow
- 3 Express creates a Reservation record with status = pending, in queue order
- 4 When a copy becomes available (another member returns it), Express checks the Reservation queue

- 5 The earliest pending reservation is notified and given priority to borrow before the book is opened to general availability